Trees and Forests

W. Evan Johnson, Ph.D.
Professor, Division of Infectious Disease
Director, Center for Data Science
Co-Director, Center for Biomedical Informatics and Health AI
Rutgers University – New Jersey Medical School

2026-06-18

WEJ LAB

Center for Data Science

Motivating example: olive oil

To motivate this section, we will use a new dataset that includes the breakdown of the composition of olive oil into 8 fatty acids:

```r
library(tidyverse)
library(dslabs)
data("olive")
olive <- select(olive, -area) #remove the `area` column--don't use it
names(olive)
```

```
## [1] "region"      "palmitic"    "palmitoleic" "stearic"     "oleic"
## [6] "linoleic"    "linolenic"   "arachidic"   "eicosenoic"
```

Motivating example: olive oil

Here is a quick look at the data:

```
olive %>% head() %>%
  knitr::kable() %>% kable_styling(font_size =7)
```

region	palmitic	palmitoleic	stearic	oleic	linoleic	linolenic	arachidic	eicosenoic
Southern Italy	10.75	0.75	2.26	78.23	6.72	0.36	0.60	0.29
Southern Italy	10.88	0.73	2.24	77.09	7.81	0.31	0.61	0.29
Southern Italy	9.11	0.54	2.46	81.13	5.49	0.31	0.63	0.29
Southern Italy	9.66	0.57	2.40	79.52	6.19	0.50	0.78	0.35
Southern Italy	10.51	0.67	2.59	77.71	6.72	0.50	0.80	0.46
Southern Italy	9.11	0.49	2.68	79.24	6.78	0.51	0.70	0.44

Motivating example: olive oil

We will try to predict the region using the fatty acid composition values as predictors.

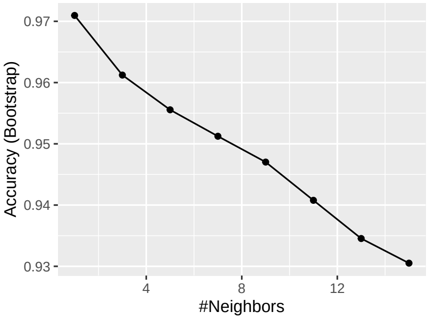
```
table(olive$region)
```

```
##
## Northern Italy        Sardinia Southern Italy
##            151              98            323
```

Motivating example: olive oil

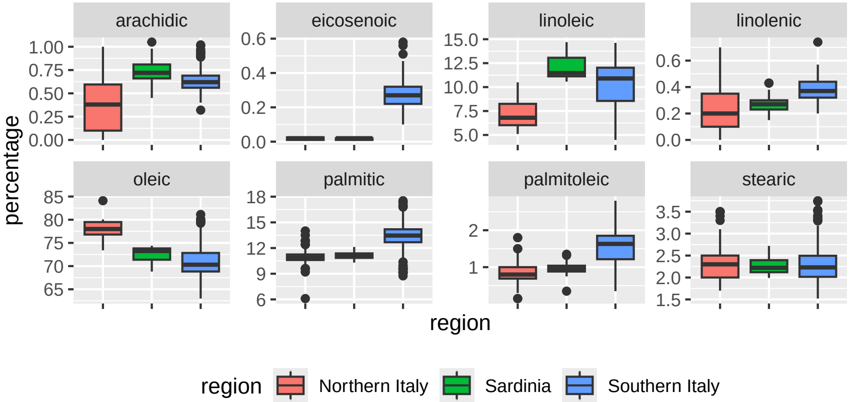
Let's very quickly try to predict the region using kNN:

```
library(caret)
fit <- train(region ~ .,  method = "knn", data = olive,
             tuneGrid = data.frame(k = seq(1, 15, 2)))
ggplot(fit)
```



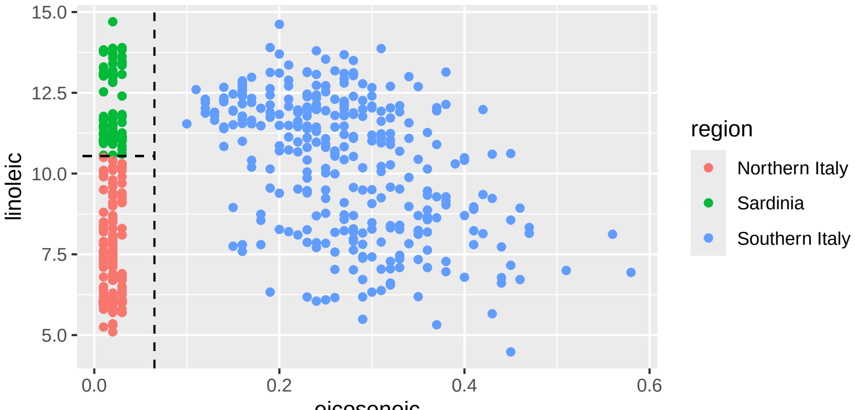
Motivating example: olive oil

A bit of data exploration reveals that we should be able to do even better: note that eicosenoic is only in Southern Italy and linoleic separates Northern Italy from Sardinia.



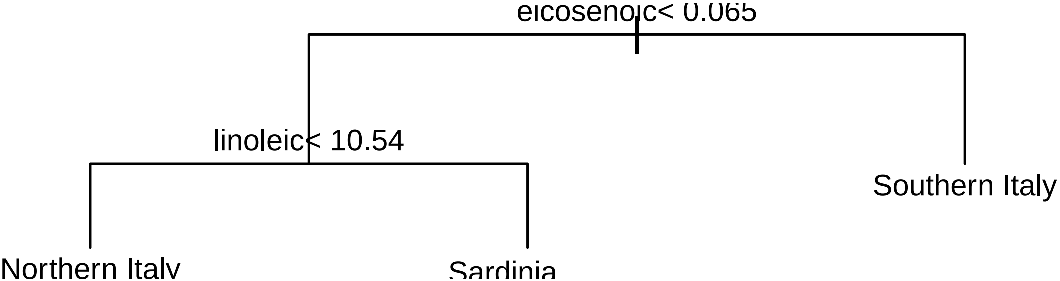
Motivating example: olive oil

This implies that we should be able to build an algorithm that predicts perfectly! Let's try plotting the values for eicosenoic and linoleic.



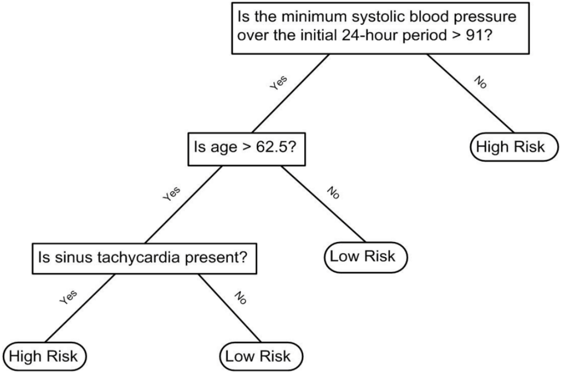
Motivating example: olive oil

Let's define a **decision rule**: If eicosenoic is larger than 0.065, predict Southern Italy. If not, then if linoleic is larger than 10.535, predict Sardinia, otherwise predict Northern Italy. We can draw this decision tree:



Decision Trees

Decision trees like this are often used in practice. For example, to evaluate a person's risk of poor outcome after a heart attack, doctors may use:



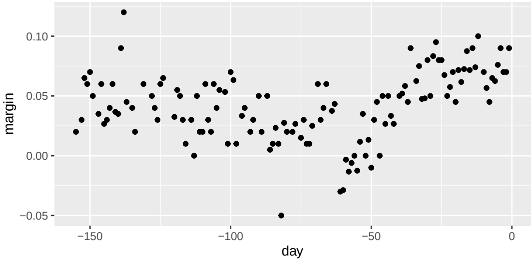
Decision Trees

- A tree is a **flow chart of yes/no questions**, with predictions at the ends (**nodes**)
- **CART** (Classification and Regression Trees) predicts an outcome Y by **partitioning the predictors** $\mathbf{X}$

Regression trees

For continuous outcomes, we call the method a **regression tree**. We use polling data to estimate the conditional expectation $f(x) = \mathrm{E}(Y|X = x)$ with Y the poll margin and x the day.

```
data("polls_2008")
polls_2008 %>% ggplot() + geom_point(aes(day, margin))
```



Regression trees

- Build a tree; each **node** yields a prediction $\hat{y}$
- Partition the predictor space into J non-overlapping regions $R_1, R_2, \ldots, R_J$
- For any $x \in R_j$, estimate $f(x)$ by the **average of the training** y_i in R_j

Regression trees

Partitions are created **recursively**:

- Start with one partition — the whole predictor space (here, the interval $[-155, 1]$)
- Each step splits one partition in two: 2 partitions, then 3, 4, 5, ...

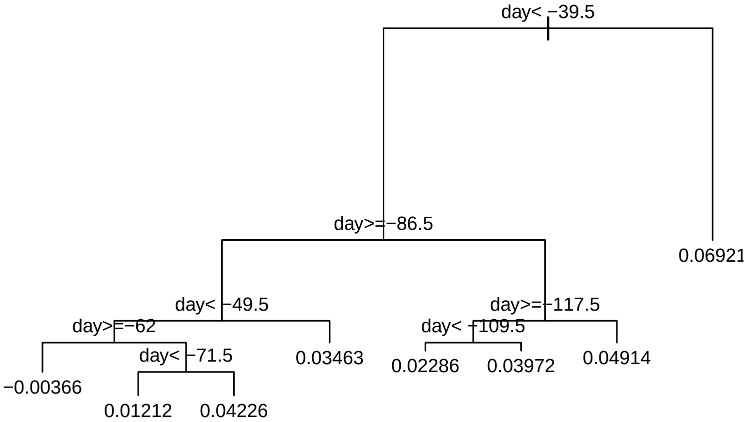
Regression trees

Let's take a look at what this algorithm does on the 2008 presidential election poll data. We will use the `rpart` function in the **rpart** package.

```
library(rpart)
fit <- rpart(margin ~ ., data = polls_2008)
```

Regression trees

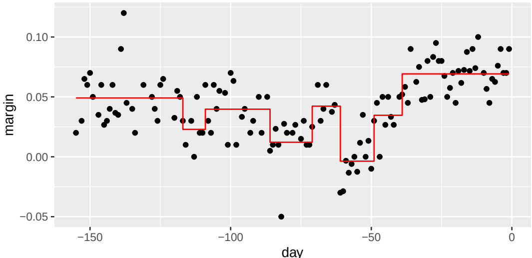
We can visually see where the splits were made:



Regression trees

The first split is made on day -39.5, then split at day -86.5. The two resulting new partitions are split on days -49.5 and -117.5, respectively, and so on. The final estimate $\hat{f}(x)$ looks like this:

```
polls_2008 %>%
  mutate(y_hat = predict(fit)) %>% ggplot() +
  geom_point(aes(day, margin)) + geom_step(aes(day, y_hat), col="red")
```



Regression trees

- Each split **lowers training RSS** (more flexibility to fit the data)
- Split until every point is its own partition $\rightarrow$ RSS $= 0$ (**overfitting**)
- To prevent this: require a **minimum RSS improvement** per split — the **complexity parameter (cp)**

Regression trees

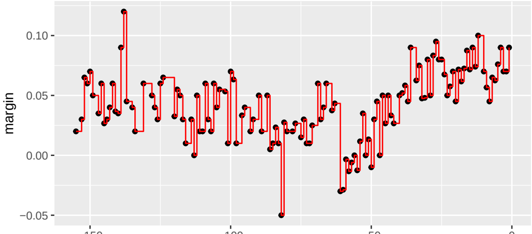
Other `rpart` stopping parameters:

- **`minsplit`** — min observations needed to split a node (default 20)
- **`minbucket`** — min observations in each leaf (default `round(minsplit/3)`)

Regression trees

If we set `cp = 0` and `minsplit = 2`, then our prediction is as flexible as possible and our predictor is our original data:

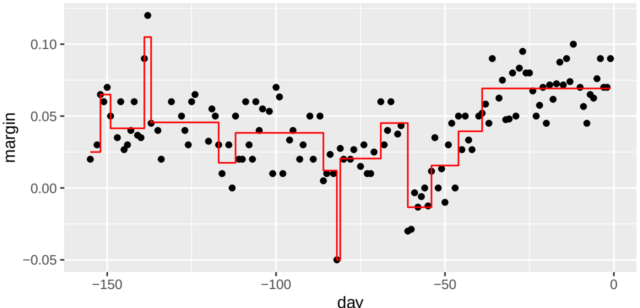
```
fit <- rpart(margin ~ ., data = polls_2008,
             control = rpart.control(cp = 0, minsplit = 2))
polls_2008 %>%
  mutate(y_hat = predict(fit)) %>% ggplot() +
  geom_point(aes(day, margin)) +  geom_step(aes(day, y_hat), col="red")
```



Regression trees

If we already have a tree and want to apply a higher cp value, we can **prune** the tree with the `prune` function:

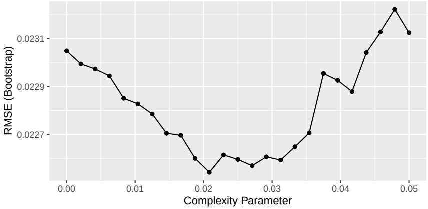
```
pruned_fit <- prune(fit, cp = 0.01)
polls_2008 %>% mutate(y_hat = predict(pruned_fit)) %>% ggplot() +
  geom_point(aes(day, margin)) + geom_step(aes(day, y_hat), col="red")
```



Regression trees

But how do we pick these parameters? We can use cross validation! Here is an example of using cross validation to choose cp:

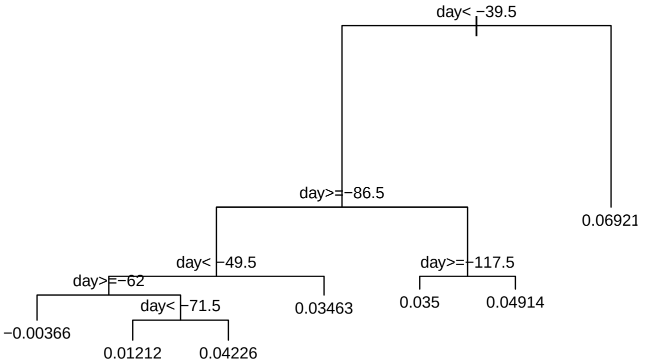
```
library(caret)
train_rpart <- train(margin ~ ., method = "rpart",
                     tuneGrid = data.frame(cp = seq(0, 0.05, len = 25)),
                     data = polls_2008)
ggplot(train_rpart)
```



Regression trees

To see the resulting tree, we access the `finalModel` and plot it:

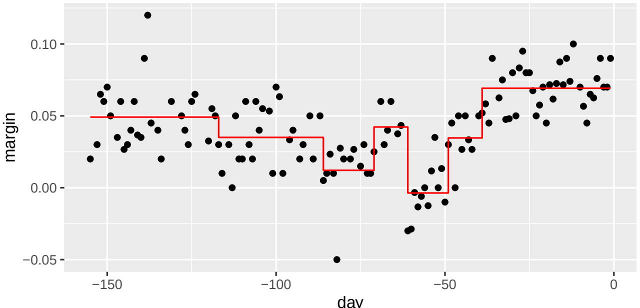
```
plot(train_rpart$finalModel, margin = 0.1)
text(train_rpart$finalModel, cex = 0.75)
```



Regression trees

And because we only have one predictor, we can actually plot $\hat{f}(x)$:

```
polls_2008 %>%
  mutate(y_hat = predict(train_rpart)) %>% ggplot() +
  geom_point(aes(day, margin)) +
  geom_step(aes(day, y_hat), col="red")
```



Classification (decision) trees

Used when the **outcome is categorical**. Two changes from regression trees:

- **Predict** the *most common class* in each partition (not the average)
- **Split** using metrics like the **Gini Index** or **Entropy**

Classification (decision) trees

Let $\hat{p}_{j,k}$ be the proportion of class k in partition j. The **Gini Index** is

$$\text{Gini}(j) = \sum_{k=1}^{K} \hat{p}_{j,k}(1 - \hat{p}_{j,k}).$$

- 0 when a partition is pure (one class) — the ideal
- Larger as the classes get more mixed

Classification (decision) trees

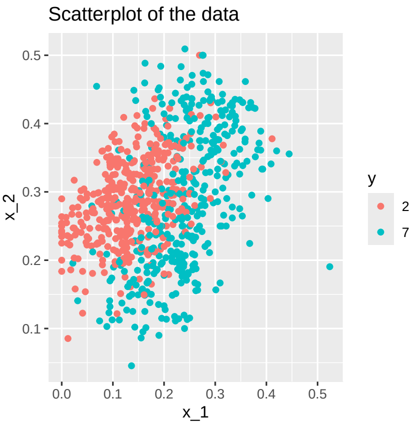
The **Entropy** is defined as

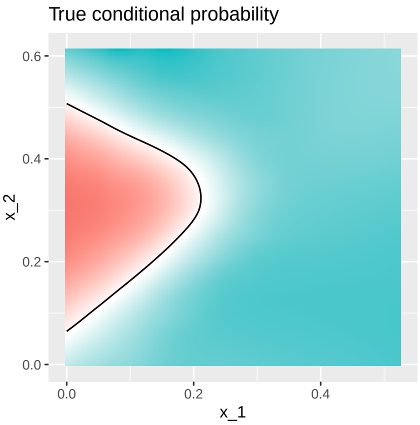
$$\text{entropy}(j) = -\sum_{k=1}^{K} \hat{p}_{j,k} \log(\hat{p}_{j,k}),$$

with $0 \times \log(0)$ defined as 0.

Classification (decision) trees

Lets look at how a classification tree performs on the following dataset:

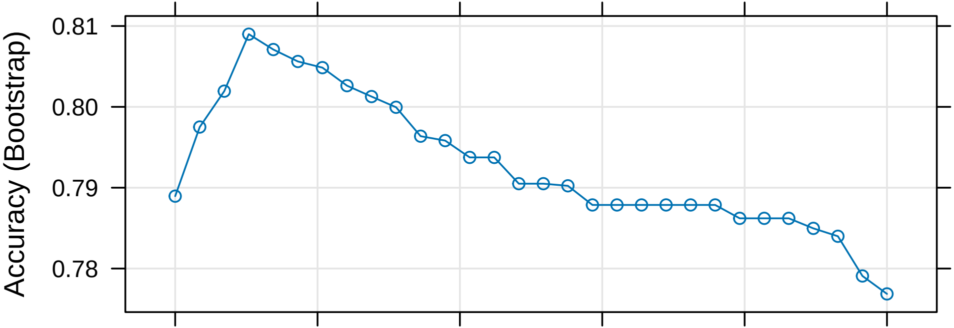




Classification (decision) trees

Trying classification trees with different complexities:

```
train_rpart <- train(y ~ ., method = "rpart", data = mnist_27$train,
                     tuneGrid = data.frame(cp = seq(0.0, 0.1, len = 30)))
plot(train_rpart)
```



Center for Data Science

Classification (decision) trees

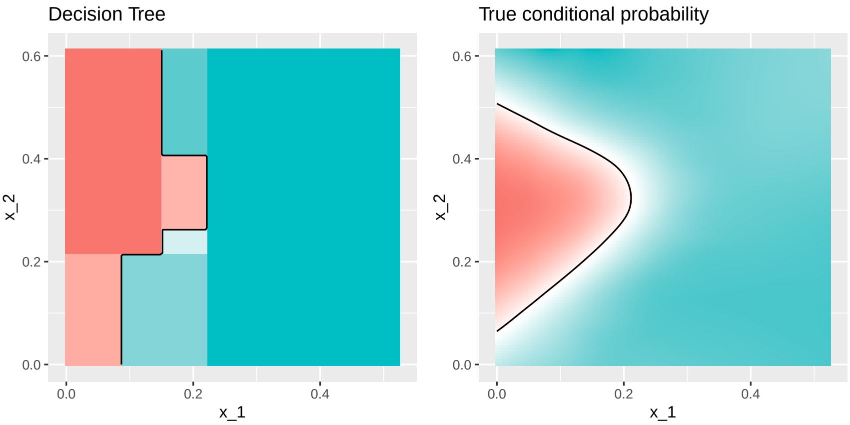
The accuracy achieved by this approach is better than what we got with regression, but is not as good as what we achieved with kernel methods:

```
y_hat <- predict(train_rpart, mnist_27$test)
confusionMatrix(y_hat, mnist_27$test$y)$overall["Accuracy"]
```

```
## Accuracy
##     0.81
```

Classification (decision) trees

The conditional probability shows us the limitations of classification trees:



Classification (decision) trees

Classification trees have certain advantages that make them very useful.

- They are highly interpretable, even more so than linear models.
- They are easy to visualize (if small enough).
- They can model human decision processes and don't require use of dummy predictors for categorical variables.

On the other hand:

- The approach via recursive partitioning can easily over-train and is therefore a bit harder to train than, for example, linear regression or kNN.
- In terms of accuracy, it is rarely the best performing method since it is not very flexible and is highly unstable to changes in training data.

Random forests, explained next, improve on several of these shortcomings.

Random forests

Random forests fix the instability of single decision trees by **averaging many trees**:

- **Bagging** (bootstrap aggregation): build many regression/classification trees on bootstrap samples
- Combine them — **average** (regression) or **majority vote** (classification)
- The bootstrap makes trees randomly different; the set of trees is the "**forest**"

Random forests

The specific steps are as follows.

1. Build B decision trees, $T_1, T_2, \ldots, T_B$. using the training set. We later explain how we ensure they are different.

2. For every observation in the test set, form a prediction $\hat{y}_j$ using tree T_j.

3. For continuous outcomes, form a final prediction with the average $\hat{y} = \frac{1}{B}\sum_{j=1}^{B}\hat{y}_j$. For categorical data classification, predict $\hat{y}$ with majority vote (most frequent class among $\hat{y}_1, \ldots, \hat{y}_T$).

Random forests

We use randomness in two ways to get different trees. Let N be the number of observations in the training set. To create T_j, $j = 1, \ldots, B$ we do the following:

1. Create a bootstrap training set by **sampling** N **observations** from the training set *with replacement*. This is the first way to induce randomness.

2. **Randomly selecting predictor variables**, $\mathbf{x}$, to be included (or excluded) in the building of each tree. A different random subset is selected for each tree.

WEJ LAB

Center for Data Science

Random forests

We will demonstrate how this works by fitting a random forest to the 2008 polls data. We will use the **randomForest** package:

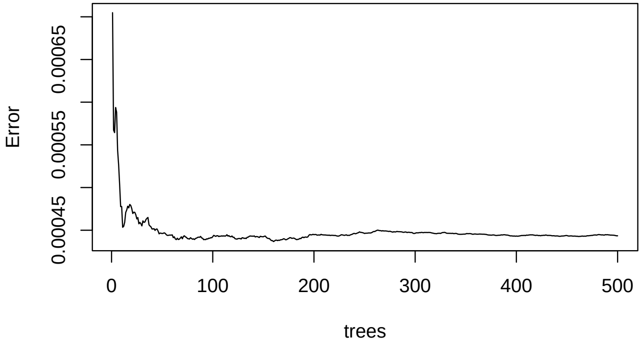
```
library(randomForest)
fit <- randomForest(margin~., data = polls_2008)
fit
```

```
##
## Call:
##  randomForest(formula = margin ~ ., data = polls_2008)
##                Type of random forest: regression
##                      Number of trees: 500
## No. of variables tried at each split: 1
##
##           Mean of squared residuals: 0.0004434732
##                     % Var explained: 46.87
```

Random forests

Note that if we apply the function `plot` to the resulting object, we see how the error rate of our algorithm changes as we add trees.

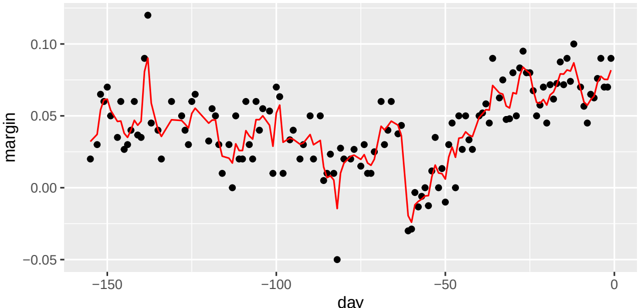
```
plot(fit,main='')
```



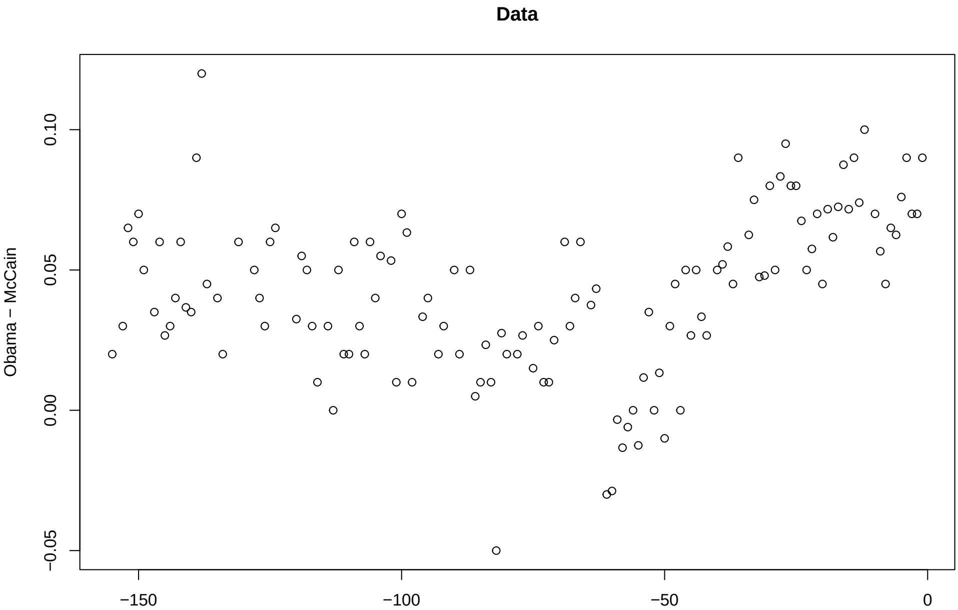
Random forests

In this case, the accuracy improves as we add more trees until about 30 trees where accuracy stabilizes. The results for this random forest looks like this:

```
polls_2008 %>%
  mutate(y_hat = predict(fit, newdata = polls_2008)) %>% ggplot() +
  geom_point(aes(day, margin)) + geom_line(aes(day, y_hat), col="red")
```



Here we see each of the bootstrap samples for several values of b:



Random forests

Here is the random forest fit for our digits example based on two predictors:

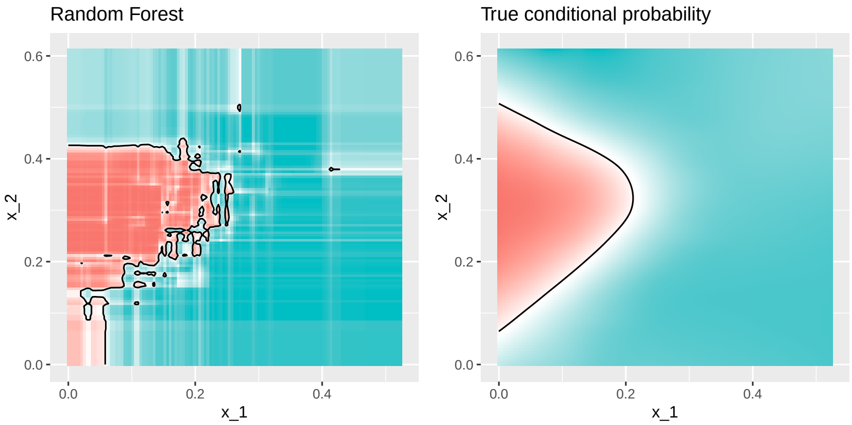
```r
library(randomForest)
train_rf <- randomForest(y ~ ., data=mnist_27$train)

confusionMatrix(predict(train_rf, mnist_27$test),
                mnist_27$test$y)$overall["Accuracy"]
```

```
## Accuracy
##     0.82
```

Random forests

Here is what the conditional probabilities look like:



Random forests

High accuracy, but the estimate can be made **smoother**:

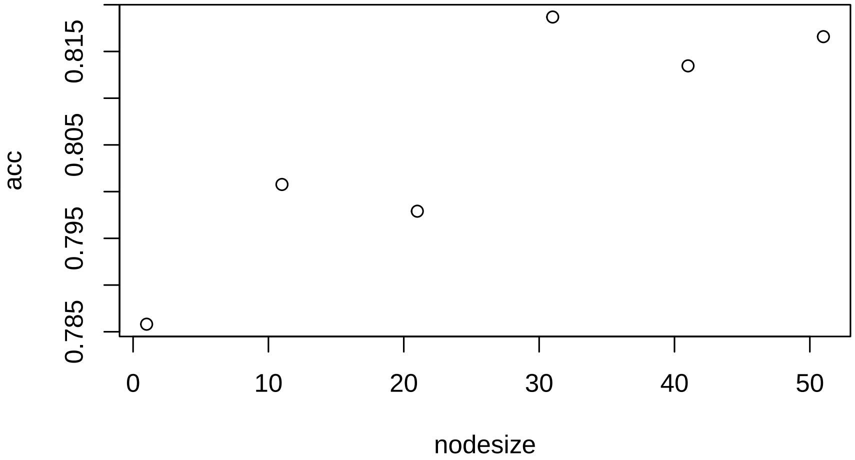
- Controlled by the **minimum node size** (data points per leaf)
- Larger minimum $\rightarrow$ smoother estimate
- This parameter can be **tuned**

Random forests

We can use the **caret** package to optimize over the minimum node size:

```
nodesize <- seq(1, 51, 10)
acc <- sapply(nodesize, function(ns){
  train(y ~ ., method = "rf", data = mnist_27$train,
        tuneGrid = data.frame(mtry = 2),
        nodesize = ns)$results$Accuracy })
plot(nodesize, acc)
```

Random forests



Random forests

We can now fit the random forest with the optimized minimum node size to the entire training data and evaluate performance on the test data.

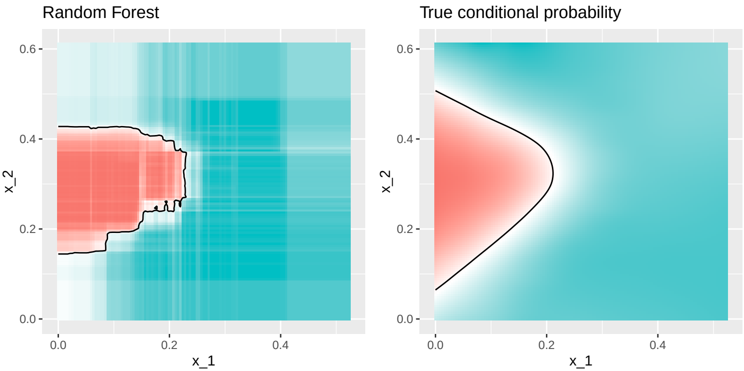
```r
train_rf_2 <- randomForest(y ~ ., data=mnist_27$train,
                           nodesize = nodesize[which.max(acc)])

confusionMatrix(predict(train_rf_2, mnist_27$test),
                mnist_27$test$y)$overall["Accuracy"]
```

```
## Accuracy
##     0.84
```

Random forests

The selected model improves accuracy and provides a smoother estimate.



Random forests

- Other implementations exist (see the **caret manual**)
- Random forests perform well, but **lose interpretability**
- **Variable importance** helps: count how often each predictor is used across trees (details here)
- `caret::varImp()` extracts it from any supported model

Session Info

```r
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.5.1
##
## Matrix products: default
## BLAS:   /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/libBLAS.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib;  LAPACK version 3.12.1
##
## locale:
## [1] en_US/en_US/en_US/C/en_US/en_US
##
## time zone: America/New_York
## tzcode source: internal
##
## attached base packages:
## [1] stats     graphics  grDevices utils     datasets  methods   base
##
## other attached packages:
##  [1] randomForest_4.7-1.2 rpart_4.1.27         dslabs_0.9.1
##  [4] gridExtra_2.3        kableExtra_1.4.0     lubridate_1.9.5
##  [7] forcats_1.0.1        stringr_1.6.0        dplyr_1.2.1
## [10] purrr_1.2.2          readr_2.2.0          tidyr_1.3.2
## [13] tibble_3.3.1         tidyverse_2.0.0      caret_7.0-1
## [16] lattice_0.22-9       ggplot2_4.0.3
##
```